

# Funcionamento

O sistema é feito em C/C++ e realiza o processamento de quadros de vídeo em tempo real. Ele permite ao usuário aplicar diferentes filtros a um arquivo de vídeo especificado na inicialização do programa.

## Fluxo de Uso:

Compilar - `mpic++ -fopenmp -o mainProc mainProc.cpp $(pkg-config --cflags --libs opencv4)`

Executa o programa via terminal passando o caminho do arquivo de vídeo como argumento (ex: `./mainProc video2.avi`).

O sistema exibe um menu interativo no terminal com as opções de filtros disponíveis.

O usuário seleciona o filtro desejado e, dependendo do filtro, informa a intensidade do efeito.

O sistema abre uma janela gráfica reproduzindo o vídeo com o filtro aplicado quadro a quadro.

O usuário pode pressionar a tecla ESC para fechar o vídeo e retornar ao menu principal, ou escolher 0 no menu para encerrar a aplicação.

## Filtros Disponíveis:

Escala de Cinza: Converte a imagem colorida para tons de cinza.

Sharpen: Realça os detalhes e bordas da imagem (aumento de nitidez).

Emboss: Cria um efeito de alto-relevo tridimensional.

Sobel: Destaca as bordas e contornos da imagem calculando o gradiente espacial (aplica convolução em duas etapas).

# Interface e Telas

## Terminal (Menu Principal)

```
===== FILTROS =====  
1 - Escala de Cinza  
2 - Sharpen  
3 - Emboss  
4 - Sobel  
0 - Sair  
Opcao: |
```

Feito por meio de um while com ifs que chamam a função de processar video mandando o filtro selecionado pelo usuário

```
while (true)
{
    MPI_Barrier(MPI_COMM_WORLD);

    int opcao = -1;
    int intensidadeDoEfeito = 5;

    if (rank == 0)
    {
        while (opcao == -1)
        {
            printf("\n===== FILTROS =====\n");
            printf("1 - Escala de Cinza\n");
            printf("2 - Sharpen\n");
            printf("3 - Emboss\n");
            printf("4 - Sobel\n");
            printf("0 - Sair\n");
            printf("Opcao: ");
            fflush(stdout);

            if (scanf("%d", &opcao) != 1)
            {
                printf("\nEntrada inválida! Digite um número.\n");
                limparBufferEntrada();
                opcao = -1;
                continue;
            }

            if (opcao == 0)
            {
                printf("\nEncerrando o programa...\n");
            }
            else if (opcao < 1 || opcao > 4)
            {
                printf("\nOpção inválida! Tente novamente.\n");
                opcao = -1;
            }
            else if (opcao != 1)
            {
                printf("\nQual intensidade? ");
                fflush(stdout);

                if (scanf("%d", &intensidadeDoEfeito) != 1)
                {
                    printf("\nIntensidade inválida, usando padrão (5).\n");
                    limparBufferEntrada();
                    intensidadeDoEfeito = 5;
                }
            }
        }
    }
}
```

## Tela de Exibição do “Vídeo”

Após a seleção, uma janela é aberta pelo OpenCV. Nela, os quadros do vídeo original são substituídos em tempo real pelos quadros processados com o filtro e intensidade escolhidos. Pressionar a tecla ESC fecha esta janela e reabre as opções no terminal.

```
imshow("Video (multiplos frames mostrados em sequencia)", frame);
```



# Lógica de Funcionamento e Funções Principais

A lógica central do sistema reside na aplicação de Kernels sobre a matriz de pixels do vídeo.

## Função de Convolução

Ela percorre a imagem aplicando a matriz 3x3 (kernel) fornecida. O uso do `pragma omp parallel for` é para que as linhas da imagem sejam processadas de forma paralela por diferentes threads do processador.

```
void aplicarConvolucao(Mat& frame, const float kernel[3][3])
{
    Mat resultado = frame.clone();

    #pragma omp parallel for
    for (int y = 1; y < frame.rows - 1; y++)
    {
        const uchar* linhaCima = frame.ptr<uchar>(y - 1);
        const uchar* linhaMeio = frame.ptr<uchar>(y);
        const uchar* linhaBaixo = frame.ptr<uchar>(y + 1);

        uchar* dst = resultado.ptr<uchar>(y);

        for (int x = 1; x < frame.cols - 1; x++)
        {
            for (int c = 0; c < 3; c++)
            {
                int idx = x * 3 + c;

                float valor = 0;

                valor += linhaCima[idx - 3] * kernel[0][0];
                valor += linhaCima[idx] * kernel[0][1];
                valor += linhaCima[idx + 3] * kernel[0][2];

                valor += linhaMeio[idx - 3] * kernel[1][0];
                valor += linhaMeio[idx] * kernel[1][1];
                valor += linhaMeio[idx + 3] * kernel[1][2];

                valor += linhaBaixo[idx - 3] * kernel[2][0];
                valor += linhaBaixo[idx] * kernel[2][1];
                valor += linhaBaixo[idx + 3] * kernel[2][2];

                dst[idx] = (uchar)std::clamp((int)valor, 0, 255);
            }
        }

        resultado.copyTo(frame);
    }
}
```

## Filtro com uso de convolução e Kernels

```
void aplicarSharpen(Mat& frame, int intensidade)
{
    float kernel[3][3] =
    {
        { 0, -1, 0 },
        { -1, 4.0f + (float)intensidade, -1 },
        { 0, -1, 0 }
    };

    aplicarConvolucao(frame, kernel);
}

void aplicarEmboss(Mat& frame, int i)
{
    float kernel[3][3] =
    {
        {-2 * (float)i, -1 * (float)i, 0},
        {-1 * (float)i, 1, 1 * (float)i},
        {0, 1 * (float)i, 2 * (float)i}
    };

    aplicarConvolucao(frame, kernel);
}
```

```
void aplicarSobel(Mat& frame, int i)
{
    float kernelGx[3][3] =
    {
        {-1 * (float)i, 0, 1 * (float)i},
        {-2 * (float)i, 0, 2 * (float)i},
        {-1 * (float)i, 0, 1 * (float)i}
    };

    float kernelGy[3][3] =
    {
        {1 * (float)i, 2 * (float)i, 1 * (float)i},
        {0, 0, 0},
        {-1 * (float)i, -2 * (float)i, -1 * (float)i}
    };

    Mat gx = frame.clone();
    Mat gy = frame.clone();

    aplicarConvolucao(gx, kernelGx);
    aplicarConvolucao(gy, kernelGy);

    int total = frame.rows * frame.cols * frame.channels();
    uchar* pFrame = frame.ptr<uchar>(0);
    uchar* pGx = gx.ptr<uchar>(0);
    uchar* pGy = gy.ptr<uchar>(0);

    #pragma omp parallel for
    for (int idx = 0; idx < total; idx++)
    {
        float vx = (float)pGx[idx];
        float vy = (float)pGy[idx];
        float mag = std::sqrt(vx * vx + vy * vy);
        pFrame[idx] = (uchar)std::clamp((int)mag, 0, 255);
    }
}
```

Cada filtro dinâmico chama a função de convolução passando uma matriz específica (kernel). O filtro Sharpen, por exemplo, utiliza o valor negativo nos vizinhos diretos e um valor central baseado na variável intensidade

O filtro Sobel é diferente porque aplica duas convoluções seguidas.

## Conversão para Escala de Cinza

```
void aplicarEscalaDeCinza(Mat& frame)
{
    int canais = frame.channels();

    #pragma omp parallel for
    for (int y = 0; y < frame.rows; y++)
    {
        uchar* px = frame.ptr<uchar>(y);

        for (int x = 0; x < frame.cols; x++)
        {
            int idx = x * canais;

            uchar b = px[idx];
            uchar g = px[idx + 1];
            uchar r = px[idx + 2];

            uchar cinza = (uchar)((114 * b + 587 * g + 299 * r) / 1000);

            px[idx] = cinza;
            px[idx + 1] = cinza;
            px[idx + 2] = cinza;
        }
    }
}
```

A função de escala de cinza não usa convolução, mas manipulação direta dos canais BGR (Blue, Green, Red). A conversão utiliza a fórmula de luminosidade humana percebida. Utiliza a opção de pragma omp parallel for para paralelizar o for em múltiplas threads

## Implementação do MPI

O sistema utiliza o padrão Master/Worker para paralelizar o processamento dos quadros. O processo de rank 0 atua como o Master, que é responsável por ler o vídeo e coordenar a

distribuição do trabalho. Os outros processos atuam como Worker, sendo responsáveis por aplicar o filtro na porção de imagem recebida.

## Inicialização

O ambiente MPI é inicializado na função main, onde é realizada a identificação do rank de cada processo e do número total de processos. Em seguida, são realizadas as validações de argumentos de entrada, para garantir que a execução possua pelo menos dois processos.

```
int main(int argc, char** argv)
{
    MPI_Init(&argc, &argv);

    int rank;
    int size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("\nProcesso %d de %d inicializado.\n", rank, size);

    if (argc < 2)
    {
        if (rank == 0)
        {
            printf("\nUso: %s <video> Ex: ./mainProc video2.avi\n", argv[0]);
        }
        MPI_Finalize();
        return 0;
    }

    if (size < 2)
    {
        if (rank == 0)
        {
            printf("\n Execute com pelo menos 2 processos.\n");
            printf("Ex: mpirun -np 4 ./mainProc video2.avi\n");
        }
        MPI_Finalize();
        return 0;
    }
}
```

Todo o restante da execução, exibição do menu, processamento do vídeo e comunicação entre Master e Workers, acontece dentro de um laço externo (while (true)), que é reiniciado sempre que o vídeo termina ou o usuário pressiona ESC, retornando ao menu. O laço só é interrompido quando o usuário escolhe encerrar o programa ou quando o vídeo não pode ser aberto. Logo no início de cada repetição, é utilizada uma barreira para sincronizar todos os processos antes da exibição do menu de opções.

## Sincronização dos processos

Após a definição dos parâmetros de processamento, é realizado um broadcast para distribuir a opção de filtro e a intensidade selecionadas a todos os processos. Em seguida, é verificado se o usuário optou por encerrar o programa; nesse caso, o laço externo é

interrompido em todos os processos. Caso contrário, cada processo exibe a configuração recebida e é iniciado o processo de Master/Worker. O rank 0 realiza a leitura e distribuição dos quadros do vídeo, enquanto os outros processos permanecem em espera para receber e processar os blocos de imagem enviados pelo Master. Caso o Master não consiga abrir o vídeo, essa falha também é usada para interromper o laço de execução em todos os processos.

```
MPI_Bcast(&opcao, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&intensidadeDoEfeito, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (opcao == 0)
{
    break;
}

printf("\nProcesso %d recebeu configuração -> Filtro: %d | Intensidade: %d\n", rank, opcao, intensidadeDoEfeito);

if (rank == 0)
{
    printf("\nProcesso Master iniciando processamento do vídeo...\n");
    if (!master(argv[1], opcao, intensidadeDoEfeito, size))
    {
        break;
    }
}
else
{
    printf("Processo Worker %d aguardando blocos de imagem...\n", rank);
    worker(rank, opcao, intensidadeDoEfeito);
}
```

## Master

É iniciada a função Master abrindo o arquivo do vídeo e verificando se o vídeo foi carregado corretamente. Caso ocorra alguma falha, é utilizado um broadcast para sinalizar aos outros processos que o processamento não poderá ser realizado, e a função retorna um valor booleano indicando o insucesso ao processo principal. Caso não haja falhas, continua para o while que controla a leitura do vídeo.

```
bool master(const char* nomeArquivo, int filtro, int intensidade, int size)
{
    VideoCapture capture(nomeArquivo);

    if (!capture.isOpened())
    {
        printf("\nErro ao abrir o vídeo.\n");
        int fimErro[3] = {0, 0, 0};
        MPI_Bcast(fimErro, 3, MPI_INT, 0, MPI_COMM_WORLD);
        return false;
    }

    printf("\nVídeo aberto com sucesso.\n");

    Mat frame;
```

A cada quadro lido, é verificado se os dados da imagem estão armazenados de forma contígua em memória, condição necessária para o envio correto dos blocos por linha; caso não estejam, uma cópia contígua é criada. Em seguida, o Master obtém as informações da



imagem, como largura, altura e número de canais, e as distribui para todos os processos através de um broadcast. Essas informações são necessárias para que cada Worker consiga criar corretamente a estrutura da imagem que será processada.

```
while (capture.read(frame))
{
    if (!frame.isContinuous())
    {
        frame = frame.clone();
    }

    int largura = frame.cols;
    int altura = frame.rows;
    int canais = frame.channels();

    int info[3];
    info[0] = largura;
    info[1] = altura;
    info[2] = canais;

    MPI_Bcast(info, 3, MPI_INT, 0, MPI_COMM_WORLD);

    int workers = size - 1;

    int linhasBase = altura / workers;
    int resto = altura % workers;

    int linhaInicial = 0;
```

Após obter as dimensões do quadro, o Master divide a imagem em blocos horizontais entre os Workers. Cada processo recebe um conjunto de linhas da imagem, incluindo uma linha adicional acima e abaixo (quando existirem), garantindo que os filtros de convolução possam ser aplicados corretamente nas bordas do bloco. Caso o número de linhas do vídeo seja menor que o número de Workers disponíveis, algum processo pode não receber nenhuma linha; esse caso é tratado explicitamente enviando um bloco de tamanho zero, para que o Worker correspondente permaneça inativo naquele quadro sem travar a comunicação. Em seguida, os dados são enviados aos Workers.

```

for (int destino = 1; destino < size; destino++)
{
    int linhas = linhasBase;

    if (destino <= resto)
        linhas++;

    int primeiraLinha = linhaInicial;
    int ultimaLinha = linhaInicial + linhas - 1;

    int inicioEnvio = primeiraLinha;
    int fimEnvio = ultimaLinha;

    if (linhas > 0)
    {
        if (inicioEnvio > 0)
            inicioEnvio--;

        if (fimEnvio < altura - 1)
            fimEnvio++;
    }

    int linhasEnviar = (linhas > 0) ? fimEnvio - inicioEnvio + 1 : 0;

    int dados[3];
    dados[0] = primeiraLinha;
    dados[1] = linhas;
    dados[2] = linhasEnviar;

    MPI_Send(dados, 3, MPI_INT, destino, 0, MPI_COMM_WORLD);

    if (linhasEnviar > 0)
    {
        MPI_Send(frame.ptr(inicioEnvio), linhasEnviar * largura * canais, MPI_UNSIGNED_CHAR, destino, 1, MPI_COMM_WORLD);
        printf("Master enviou %d linhas para Worker %d.\n", linhasEnviar, destino);
    }
    else
    {
        MPI_Send(nullptr, 0, MPI_UNSIGNED_CHAR, destino, 1, MPI_COMM_WORLD);
        printf("Master enviou 0 linhas para Worker %d (desligado).\n", destino);
    }

    linhaInicial += linhas;
}

```

Após o processamento paralelo realizado pelos Workers, o Master recebe de cada processo apenas as linhas que foram processadas e recompõe o quadro original. O resultado é exibido na tela e o processo se repete até o término do vídeo ou até que a tecla ESC seja pressionada. Ao final da execução, é enviado um sinal de encerramento para todos os Workers, liberando os recursos utilizados.

```

for (int origem = 1; origem < size; origem++)
{
    int linhas = linhasBase;

    if (origem <= resto)
        linhas++;

    if (linhas > 0)
    {
        MPI_Recv(frame.ptr(linhaInicial), linhas * largura * canais, MPI_UNSIGNED_CHAR, origem, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Master recebeu %d linhas do Worker %d.\n", linhas, origem);
    }
    else
    {
        MPI_Recv(nullptr, 0, MPI_UNSIGNED_CHAR, origem, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Master recebeu 0 linhas do Worker %d (desligado).\n", origem);
    }

    linhaInicial += linhas;
}

imshow("Video MPI + OpenMP", frame);

if ((waitKey(30) & 0xFF) == 27)
    break;
}

int fim[3] = {0, 0, 0};

MPI_Bcast(fim, 3, MPI_INT, 0, MPI_COMM_WORLD);

printf("Master finalizando processamento.\n");

capture.release();

destroyAllWindows();

return true;

```

## Worker

Inicialmente, o Worker recebe, através de um broadcast, as informações referentes às dimensões da imagem. Caso seja recebido um vetor de controle contendo apenas valores zero, o processo é encerrado. Caso contrário, o Worker recebe do Master as informações sobre o bloco que deverá processar e, em seguida, os dados da imagem, criando a estrutura da imagem com o número de canais correspondente ao quadro recebido. Se o bloco recebido tiver tamanho zero (caso em que há mais Workers do que linhas disponíveis naquele quadro), o Worker apenas recebe a mensagem vazia e permanece inativo até o próximo quadro.

```

void worker(int rank, int filtro, int intensidade)
{
    while (true)
    {
        int info[3];
        MPI_Bcast(info, 3, MPI_INT, 0, MPI_COMM_WORLD);

        int largura = info[0];
        int altura = info[1];
        int canais = info[2];

        if (largura == 0 && altura == 0 && canais == 0)
        {
            printf("Worker %d recebeu sinal de término.\n", rank);
            break;
        }

        int dados[3];
        MPI_Recv(dados, 3, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        int primeiraLinha = dados[0];
        int linhas = dados[1];
        int linhasEnviar = dados[2];

        Mat bloco;
        if (linhasEnviar > 0)
        {
            bloco = Mat(linhasEnviar, largura, CV_8UC(canais));
            MPI_Recv(bloco.ptr(0), linhasEnviar * largura * canais, MPI_UNSIGNED_CHAR, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
        else
        {
            MPI_Recv(nullptr, 0, MPI_UNSIGNED_CHAR, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
    }
}

```

Após receber o bloco de imagem, o Worker aplica o filtro selecionado pelo usuário. O processamento é realizado sobre o bloco recebido, utilizando a intensidade distribuída pelo processo Master. Após concluir o processamento, o Worker envia ao processo Master apenas as linhas que foram atribuídas a ele. As linhas adicionais recebidas para o cálculo da convolução são descartadas, sendo utilizadas apenas como suporte para o processamento das bordas do bloco. Então o Worker retorna ao início do laço para aguardar um novo bloco de imagem ou um sinal de encerramento.

```
if (linhas > 0)
{
    switch (filtro)
    {
        case 1: aplicarEscalaDeCinza(bloco); break;
        case 2: aplicarSharpen(bloco, intensidade); break;
        case 3: aplicarEmboss(bloco, intensidade); break;
        case 4: aplicarSobel(bloco, intensidade); break;
        default:
            printf("\nOpção inválida! Tente novamente.\n");
            break;
    }

    int offsetTopo = (primeiraLinha == 0) ? 0 : 1;
    MPI_Send(bloco.ptr(offsetTopo), linhas * largura * canais, MPI_UNSIGNED_CHAR,
             0, 2, MPI_COMM_WORLD);
    printf("Worker %d processou e enviou %d linhas de volta ao Master.\n", rank, linhas);
}
else
{
    MPI_Send(nullptr, 0, MPI_UNSIGNED_CHAR, 0, 2, MPI_COMM_WORLD);
    printf("Worker %d recebeu 0 linhas e ficou inativo.\n", rank);
}
```